Cluster Mode migration

The dashboard now runs in Elastic Beanstalk Cluster Mode, replacing the earlier Standard single-instance deployment at the owner's request. Cluster Mode provisions an HTTPS listener and an ACM certificate for its own environment domain; a custom domain is optional. HTTP redirects to HTTPS.

Verified release: 17 October 2026

- Open the HTTPS application.
- Successful OIDC workflow: security, verify and deploy passed.
- Source commit: `5a5fa3e62e8c4f2a4a6c1f2e27208fddeada490d`.
- Image digest: `sha256:3433e9812618a3d2be611680ad9cc90867421f4c3a785d894cc3108c5eef5e87`.
- Release: `release-5a5fa3e62e8c-39055100638-1`, Ready / Green.
- 65 targeted Python tests and 227 frontend tests passed.
- Trusted TLS, HTTP-to-HTTPS redirect, health and built assets verified.
- Login is enabled; a real browser submitted a nonexistent test identity through the SDK and received normal invalid-credentials validation (401). An empty same-origin request received field validation (400). No account was created.
- Replica, memory and CPU metrics contain actual CloudWatch samples; both Cluster alarms are OK. No alarm notification subscriptions are configured.

The first OIDC update rolled back cleanly after CloudFormation reported missing ListTagsForResource permission. Granting that read action on only this environment resolved the failure. The successful final run above is the delivery evidence.

Infrastructure ownership

The pinned Terraform AWS provider accepts only WebServer and Worker tiers. The current upstream provider source also lacks Cluster. Terraform therefore owns a named CloudFormation stack whose AWS::ElasticBeanstalk::Environment uses Tier Name=Cluster, Type=EKS. This avoids pretending that the unsupported native Terraform environment resource can configure Cluster Mode.

The stack reuses the account's existing cluster, node and observability roles. A separate application role uses EKS Pod Identity and can read only the two configured runtime secrets. Neither AWS keys nor secret values enter Terraform. The dashboard starts with one replica, a 250m CPU request, 512Mi memory request and 1Gi limit. Readiness and ALB health checks use /api/health. Node and cluster capacity remain service-managed. Cluster Mode adds EKS, load balancer and custom-metric costs.

Safe migration sequence

1. Register the previously scanned immutable ECR image as a Cluster application version.
2. Apply a creation-only Terraform plan while Standard continues serving the demo.
3. Verify the new endpoint's publicly trusted TLS certificate and HTTP redirect.
4. Enable login through Cluster runtime configuration, verify the sign-in dialog and upstream authentication validation without creating a test account.
5. Update GitHub's environment/stack variables and the OIDC role's scoped permissions.
6. Run the full security, verification and OIDC delivery pipeline against Cluster.
7. Remove the Standard environment, its instance profile and obsolete IAM/alarms through a reviewed Terraform plan after the replacement works.
8. Check the final Terraform plan, active environment, logs, metrics and login again.

Subsequent delivery

CI installs pinned AWS CLI 2.37.10 with a verified SHA256 because the installed Python SDK can predate ImageConfiguration. It registers the scanned image directly, then updates only the CloudFormation VersionLabel parameter using the previous template. Terraform ignores drift only for that version parameter, so its reviewed runtime configuration remains authoritative. CI verifies HTTPS, health, frontend assets and an empty sign-in request reaching authentication validation.

Login is distinct from permission to spend AWS resources. Live deployment and GitHub automatic deployment remain disabled during this migration; the application role has no customer-infrastructure provisioning permission. Do not broaden that role merely to make a trial button execute cloud operations.

Observability

The dashboard and alarms use Cluster's ElasticBeanstalk/Infrastructure namespace: EnvironmentReplicas, container_memory_working_set_bytes and the cumulative CPU counter. Replica and memory alarms replace Standard EC2 alarms. Application logs are in /aws/elasticbeanstalk/application/logs, separated by the environment's stream prefix. Shared Cluster log groups must not be deleted with an individual application. Set seven-day retention only after reviewing other users of those shared groups. Alarm notifications and application traces require further configuration.

References

- Cluster networking and HTTPS
- Cluster IAM and Pod Identity

- Cluster monitoring and metric dimensions

Scanner exceptions

The current Terraform scan passes 43 checks with 14 resource-local skips. Four release/state-bucket exceptions and one ECR exception preserve the original encryption/single-region/auditing decisions. One stack exception records that no SNS recipient is configured. Each of the four shared log groups has two explicit exceptions: AWS-managed encryption rather than customer KMS, and seven days of operational retention rather than a one-year audit archive. No blanket skip or soft-fail is used. Docker and Actions have no exceptions.

Browser testing found that InsForge SDK 1.5.2 resolves absolute /api paths, dropping a base URL's /insforge prefix. A scoped SDK fetch transport now rewrites only same-origin authentication/database calls to the proxy. Tests exercise the real SDK and ensure unrelated application routes and external URLs stay unchanged. Deployment capture tests a nonexistent example.invalid account, expects normal credential validation, and never creates an account or submits real credentials.

EBKit Git and application security

These controls extend the deployment walkthrough. Source/configuration changes here must pass CI before deployment. Repository settings and local hook installation are separate actions; committing their files does not enable them in GitHub or on another developer's laptop.

Keep private material out of Git and Docker

`.gitignore` excludes local environment files, private keys, AWS profiles, Terraform state/variables/plans and generated artifacts. `.dockerignore` excludes the same sensitive inputs from the build context. The root Dockerfile also uses specific COPY paths. Never solve a scan failure by committing a real key as a test input. Use synthetic values assembled inside tests.

Ignored files can still be force-added. The staged-path check rejects private configuration even in that case. It prints paths only, never file contents. Example environment files and Terraform variable examples remain permitted.

Already committed credentials require revocation/rotation. Deleting a file or adding an ignore rule does not invalidate a credential or remove older commits.

Choose one local commit-hook installation

Native Git hook, using Python and Gitleaks 8.30.1 on PATH:

```
git config --local core.hooksPath .githooks
```

The tracked `.githooks/pre-commit` first checks staged paths, then runs Gitleaks against the staged snapshot with redaction. Missing Gitleaks blocks the commit. It does not scan unstaged developer configuration or print secret values.

Alternatively, use the pre-commit framework:

```
python -m pip install pre-commit==4.7.0
git config --local --unset core.hooksPath
pre-commit install
pre-commit run --all-files
```

Choose this alternative only when replacing the native hook configuration. The Gitleaks hook revision is pinned to the reviewed release commit. Review `pre-commit autoupdate` changes before committing them. Local hooks can be bypassed; the full-history CI scan and protected branch are the server-side enforcement.

Secret scanning and historical test data

```
gitleaks git . --redact --config .gitleaks.toml
```

CI checks out complete history, verifies the pinned Gitleaks release archive's SHA256 digest, and scans the repository history. It keeps a redacted JSON report. The configuration extends all upstream default rules.

One reviewed historical sanitizer test used a numeric-only synthetic PAT-shaped input. `.gitleaksignore` excludes only its exact commit/path/rule/line fingerprint. No test directory, credential rule or whole commit is broadly excluded.

GitHub secret scanning and push protection provide another layer when enabled in repository settings. They complement Gitleaks; their provider/rule coverage differs.

Source, dependency and infrastructure gates

- SAST: Semgrep checks reviewed local rules for dynamic execution, shell commands, disabled TLS verification, unsafe YAML and pickle use in application/tool code. Rules stay in the repository, with metrics disabled and no registry ruleset or source upload. This small ruleset is not comprehensive taint analysis.
- SCA: `npm audit --omit=dev --audit-level=high` checks shipped JavaScript packages. `pip-audit --strict` checks the exact third-party Python versions extracted from the built runtime image with standard-library package metadata. The local EBKit distribution is covered by source checks, rather than a public package lookup. Development/scanner dependencies stay outside that runtime inventory. Trivy additionally scans the actual runtime image.
- IaC: Checkov scans Terraform, the root Dockerfile and GitHub Actions. Findings fail the security job. Any accepted exception must be narrow and explain its deployment-specific reason next to the resource; no blanket soft-fail is used.
- Container and source scanning: Trivy gates fixed HIGH/CRITICAL vulnerabilities and secret findings. Its `ignore-unfixed` policy is a coverage limit, not evidence that an unfixed vulnerability is safe. The image SBOM records dependencies.

Pinned scanner dependencies are under `security/`; Dependabot reviews them weekly, alongside app dependencies, the digest-pinned Docker base and Actions. Updating a tool can change findings and must receive the same review as application code.

Passive DAST and runtime isolation

CI runs the already scanned image in a disposable Docker network with demo mode enabled. The app has no AWS credentials, no host port publication, and no mounted developer files. The test uses a read-only root filesystem, writable temporary/job mounts, dropped capabilities, no new privileges, and CPU/memory/PID limits.

A digest-pinned official OWASP ZAP container spiders that local target and performs passive checks. Missing anti-clickjacking or content-type protections fail the gate. Other warnings remain in the JSON/HTML report for review; `-I` permits WARN results but does not permit FAIL findings or scanner errors. This is passive DAST, not an active attack scan or a test of authenticated/cloud workflows.

The server sends anti-framing, nosniff, referrer and permissions headers. CSP is still a reported warning: the application injects runtime configuration inline, so a strict policy needs nonce/hash support and integration testing. The isolated CI target uses HTTP, so HSTS there does not establish HTTPS. The Cluster deployment independently verifies its managed HTTPS endpoint.

The local baseline completed with 63 passing checks, no FAIL findings and four warning categories: missing CSP, non-storable responses, SPA coverage advice, and missing cross-origin embedder policy. These are retained for review. Cache policy and cross-origin isolation need compatibility testing; the SPA warning also means this traditional spider is not complete coverage of the React dashboard.

These Docker runtime flags are exercised in CI. The Cluster pod runtime is service-managed and requires separate platform-specific hardening validation. The live container runs nonroot and uses a multistage build. Distroless is not a drop-in runtime here: the app includes Python CLI tools, Git and Node, rather than a Node-only static service.

Branch protection, ownership and RBAC

`security/main-protection.json` is the reviewable desired policy for `main`: required PR, current `verify` and `security` checks, dismissal of stale approvals when reviews are used, resolved conversations, linear history, and no force pushes or branch deletion. Administrators are included in enforcement. At the owner's request this is a solo-maintainer policy: the required independent approval count is zero, and code-owner/last-push approval are not mandatory. When collaborators join, set the count to at least one and enable code-owner and latest-push approval.

```
python scripts/harden_github.py          # preview
python scripts/harden_github.py --apply  # change settings and save a read-back receipt
```

`.github/CODEOWNERS` initially names the real repository owner, including sensitive CI, Terraform, Docker, auth and security paths. Code-owner rules must exist on the PR's base branch to take effect. An author cannot approve their own PR; add a trusted independent reviewer before expecting an owner-authored PR to merge.

Give administrators settings access only when needed, maintainers merge/release responsibility, developers PR access, and auditors read access. Teams/custom roles depend on the account plan; do not pretend a personal repository has organization teams. Require MFA for maintainers, remove stale collaborators, review installed GitHub Apps, and keep production environment/deployment branch restrictions.

Actions start with `contents: read`. Only the protected deployment job gets `id-token: write`; PRs never receive AWS credentials. Deploy waits for both security and verification jobs and publishes the same scanned image by immutable digest.

AI scanner integration from the supplied example

The pasted Sim.ai workflow is not enabled. First confirm ownership of that workflow, what repository/PR data it retains, and who can access the result. Keep its API key in a protected CI secret store and never interpolate PR titles/bodies directly into shell source. Build a JSON payload from `GITHUB_EVENT_PATH` with a JSON parser. Use bounded timeouts, `curl --fail-with-body`, and validate the returned scan status; a successful HTTP trigger is not a completed security assessment. Fork PRs must not receive integration credentials. A reviewed GitHub App/webhook can avoid executing third-party integration credentials in untrusted PR code.

Evidence and remaining work

Keep redacted Gitleaks, Semgrep, Checkov, dependency, Trivy, ZAP and SBOM artifacts from the same source commit. Read the actual job conclusions before claiming a gate passed. The walkthrough records live deployment evidence separately from prepared configuration. A reviewed CSP, active/authenticated DAST, alarm notifications, and independent review remain distinct follow-up work until verified.

References: Gitleaks, Semgrep CLI, Checkov CLI, ZAP baseline, GitHub branch protection.

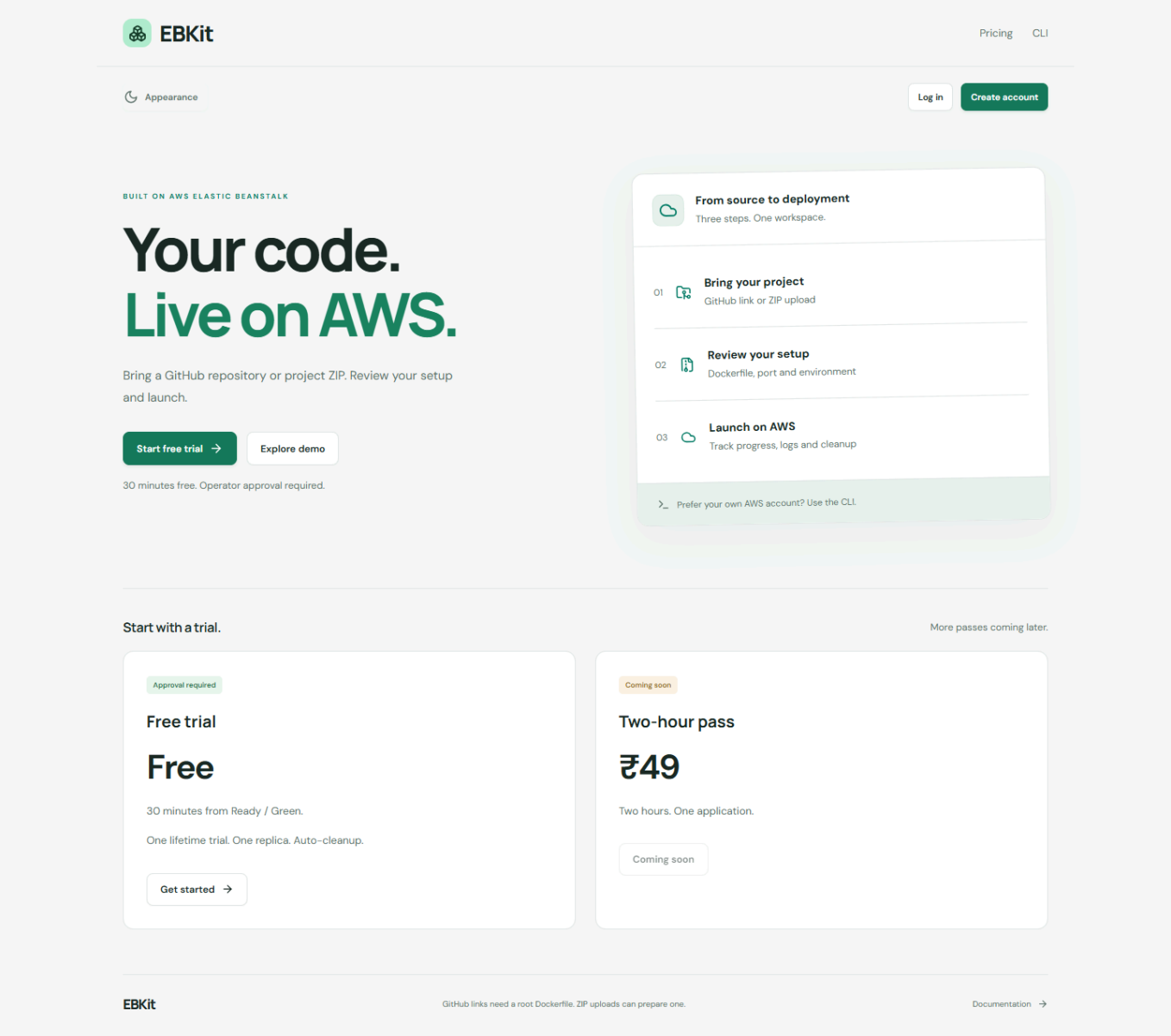
Final cutover confirmation

The Standard environment and its dedicated EC2 profile, IAM roles/policies and obsolete alarms were removed after the successful Cluster release. The reviewed Terraform cutover applied 3 additions, 5 updates and exactly 9 deletions. Only the Cluster environment remains active for this application. Both Cluster alarms are OK, all four shared log groups retain seven days, and the final full Terraform plan reports no changes.

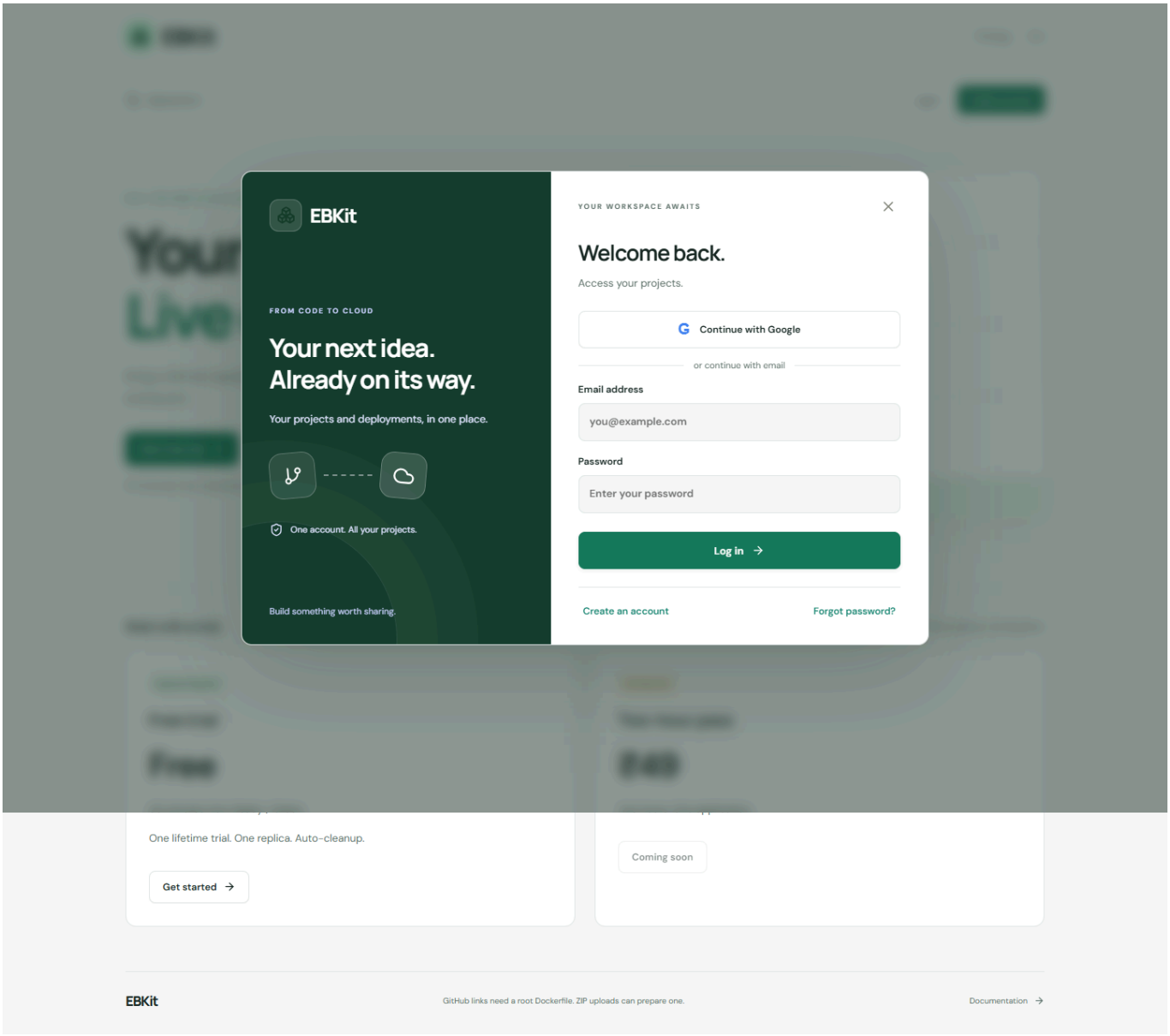
Verified release evidence

Capture time 2026-10-17T17:57:52.715323+00:00

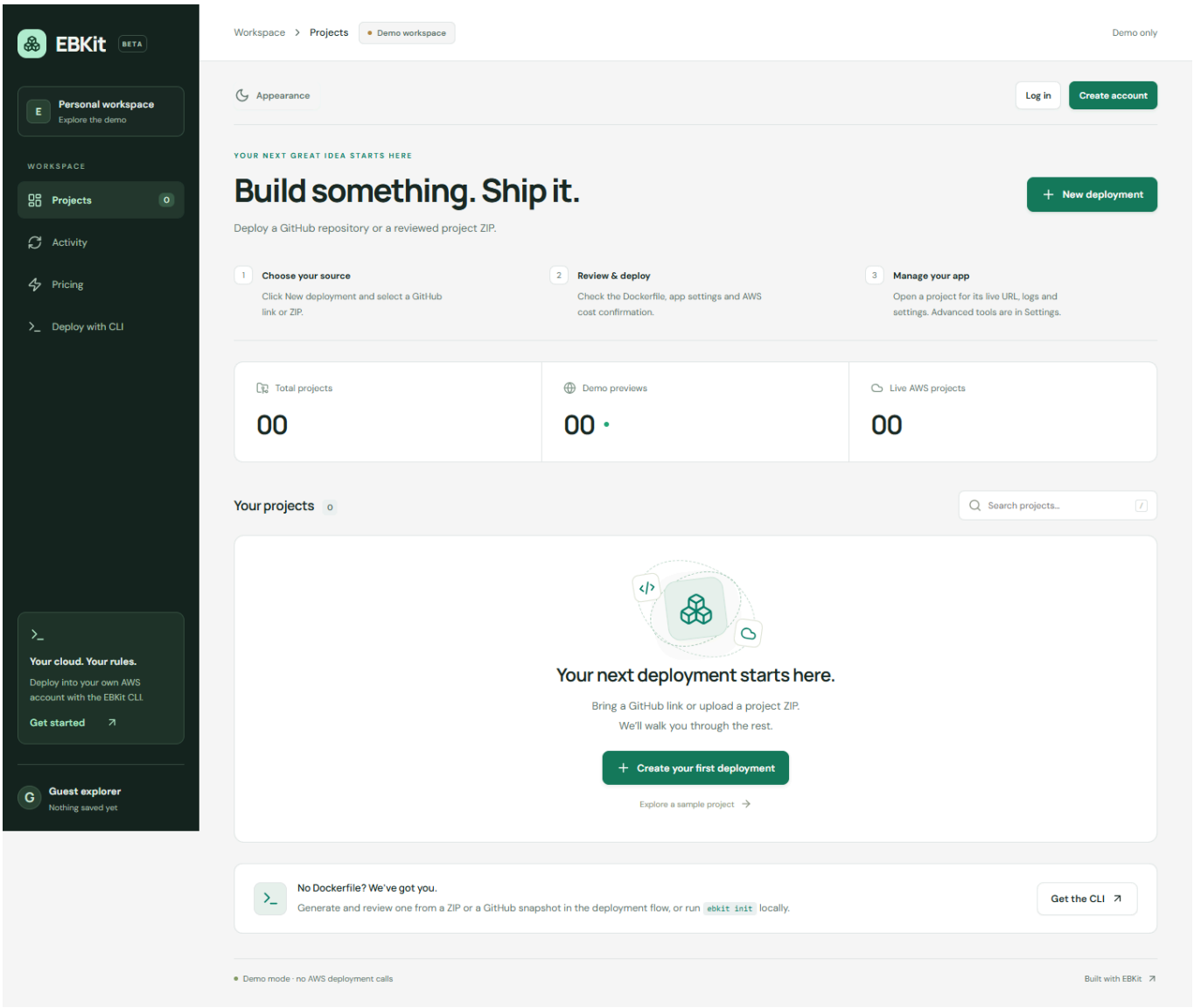

```
{
  "mode": "Cluster",
  "commit": "5a5fa3e62e8c4f2a4a6c1f2e27208fddeada490d",
  "runUrl": "https://github.com/Rakesh-Patra/ebready/actions/runs/39055100638",
  "verified": true,
  "homepageStatus": 200,
  "healthStatus": 200,
  "assetStatus": 200,
  "loginValidationStatus": 400,
  "httpRedirectsToHttps": true,
  "verifiedAt": "2026-10-17T17:53:46.063063+00:00",
  "url": "https://ebkit-dashboard-practice-cluster.eba-cktxxx2w.us-east-2.elasticbeanstalk.com",
  "awsStatus": "Ready",
  "awsHealth": "Green",
  "httpsConfigured": true,
  "imageDigest": "sha256:3433e9812618a3d2be611680ad9cc90867421f4c3a785d894cc3108c5eef5e87"
}
```



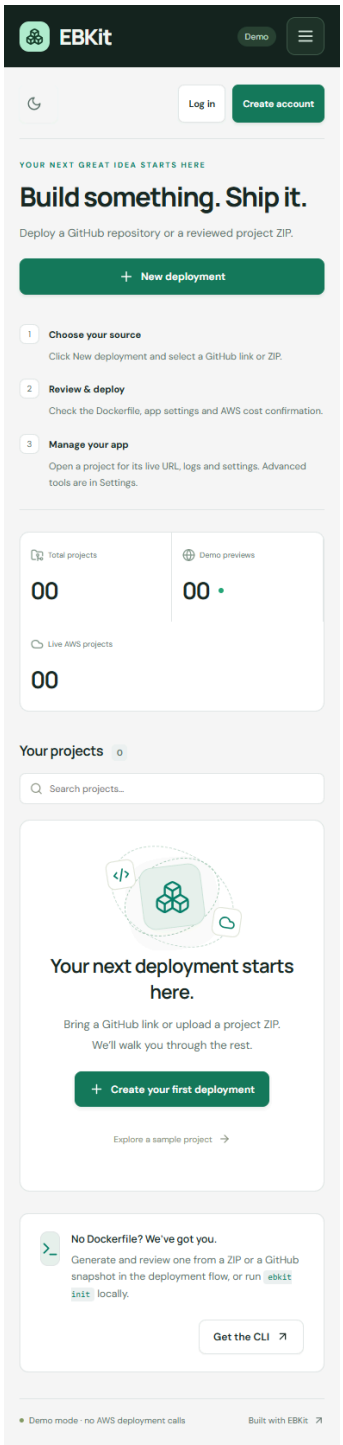
Deployed application landing page



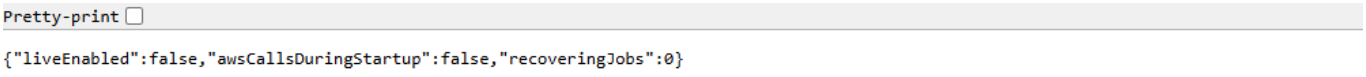
Enabled sign-in dialog on the verified HTTPS origin



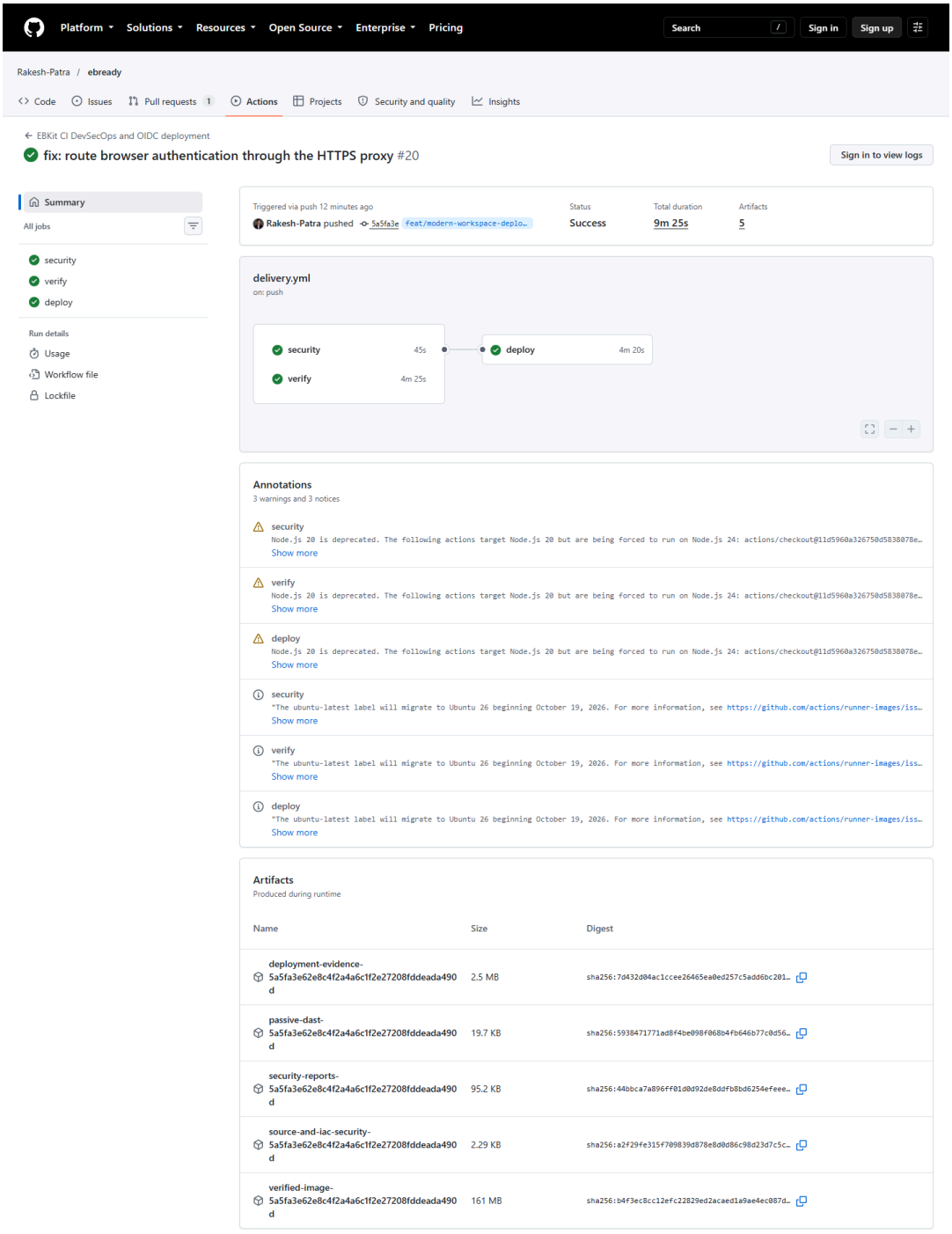
Guest dashboard served by the verified AWS release



Deployed dashboard at phone width



Health response from the deployed API



Actual public GitHub Actions run; see the linked run for individual steps

ZAP Scanning Report

Site: http://ebkit-dast:8080

Generated on Sat, 17 Oct 2026 17:49:28

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	1
Low	3
Informational	3
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP warnings logged - see the zap.log file for details	5
Info	Informational	http://ebkit-dast:8080	Percentage of responses with status code 2xx	66 %
Info	Informational	http://ebkit-dast:8080	Percentage of responses with status code 4xx	33 %
Info	Informational	http://ebkit-dast:8080	Percentage of endpoints with content type text/css	25 %
Info	Informational	http://ebkit-dast:8080	Percentage of endpoints with content type text/javascript	25 %
Info	Informational	http://ebkit-dast:8080	Percentage of endpoints with content type text/plain	50 %
Info	Informational	http://ebkit-dast:8080	Percentage of endpoints with method GET	100 %
Info	Informational	http://ebkit-dast:8080	Count of total endpoints	4

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	1
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	1
Cross-Origin-Opener-Policy Header Missing or Invalid	Low	1

Actual ZAP report for the isolated CI container; warnings require review